
TOPS Technologies | Mobile Automation Testing Assessment

Assessment Submission Mobile Automation Testing

Section A: Conceptual Questions

Q1. How does Appium support native app automation?

Appium works as a client-server framework where our test scripts send HTTP requests to the Appium server, which then passes those commands to the device using platform-specific drivers. For Android native apps, it uses the UIAutomator2 driver, and for iOS it uses XCUITest. These are Google's and Apple's own testing frameworks, so they can directly interact with native UI elements like buttons, input fields, and text views — things that a browser-based tool simply cannot reach. This is why Appium is the right choice for native app automation.

Q2. How does mobile element inspection help create reliable locators?

Appium Inspector lets us visually tap on any element in the running app and see all its attributes — like resource-id, content-desc (accessibility id), class, and text. The key is to pick locators that are stable. resource-id and accessibility id are set by developers and rarely change with UI updates, so they make tests more reliable. Locators like xpath based on element position are fragile and break easily if the layout changes. Inspection helps us understand which attribute is the safest to use for each element.

Q3. How does synchronization improve script stability on slow devices?

When a test runs on a slow device, the app takes more time to load screens or process API responses. If the script does not wait and tries to find an element that has not appeared yet, it throws a NoSuchElementException and fails. Adding Explicit Wait (WebDriverWait) tells the driver to keep checking for an element until it appears or a timeout is reached. This makes the test flexible across different device speeds instead of breaking just because the app was a second slower than expected.

Q4. Why are gestures important in mobile automation?

Mobile apps are built around touch. Users swipe to navigate, scroll to find content, and long press to open options. Many elements — like a logout button at the bottom of a settings page or an item inside a scrollable list — are simply not visible on screen unless you scroll. In desktop automation, a scroll wheel or keyboard handles this, but in mobile we have to simulate touch gestures using the Actions API. Without gesture support, large parts of many apps cannot be tested at all.

Q5. Login fails on real device but passes on emulator — reasons and approach?

This is a very common situation. The most frequent reasons are:

- The real device is slower, so elements load later and the script finds them before they are ready.
- The soft keyboard covers the Login button on a real device, but not on the emulator.
- Some real devices (especially Samsung or Xiaomi) have UI customizations that change element attributes.
- Network speed is different on real devices — login API calls take longer.

To debug this, I would first take a screenshot at the point of failure to see what is actually on screen. Then I would add explicit waits before each critical step, add `driver.hideKeyboard()` after typing, and reconnect Appium Inspector to the real device to check if the locators are still valid there.

Section B: Practical Mobile Automation Tasks

Task 1: Appium Environment Setup

1. Install Node.js (LTS version) from nodejs.org. Verify with: `node --version`
2. Install Appium globally: `npm install -g appium`, then install the driver: `appium driver install uiautomator2`
3. Install Android Studio, set up the Android SDK, and create an AVD (emulator) via AVD Manager.

4. Set environment variables: ANDROID_HOME pointing to the SDK folder, and add platform-tools to PATH.
5. Download Appium Inspector from GitHub and connect it using the Desired Capabilities below.

Sample Desired Capabilities for Android Emulator:

```
{
  "platformName": "Android",
  "deviceName": "emulator-5554",
  "automationName": "UiAutomator2",
  "platformVersion": "12.0",
  "appPackage": "com.androidsample.generalstore",
  "appActivity": "com.androidsample.generalstore.SplashActivity"
}
```

Task 2: Mobile Element Inspection — 10 Elements

#	Element	Strategy	Locator Value
1	App Logo / Header	accessibility id	app_logo
2	Username Input Field	id (resource-id)	com.example.app:id/et_username
3	Password Input Field	id (resource-id)	com.example.app:id/et_password
4	Show/Hide Password Toggle	accessibility id	toggle_password_visibility
5	Login Button	accessibility id	btn_login
6	Forgot Password Link	id (resource-id)	com.example.app:id/tv_forgot_password
7	Error Message Label	id (resource-id)	com.example.app:id/tv_error_message
8	Profile Name Header	xpath	//android.widget.TextView[@text='My Profile']
9	Remember Me Toggle	id (resource-id)	com.example.app:id/cb_remember_me
10	Logout Button	accessibility id	btn_logout

Task 3: Login Flow Pseudo-code

```
DesiredCapabilities caps = new DesiredCapabilities();
caps.setCapability("platformName", "Android");
caps.setCapability("deviceName", "emulator-5554");
caps.setCapability("automationName", "UiAutomator2");
caps.setCapability("appPackage", "com.example.loginapp");
caps.setCapability("appActivity", ".MainActivity");

AndroidDriver driver = new AndroidDriver(
    new URL("http://127.0.0.1:4723/wd/hub"), caps);
driver.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);

driver.findElement(By.id("et_username")).sendKeys("testuser@example.com");
driver.findElement(By.id("et_password")).sendKeys("Test@1234");
driver.hideKeyboard();
driver.findElement(By.id("btn_login")).click();

WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("tv_welcome")));
driver.quit();
```

Task 4: Error Message Validation

After entering wrong credentials and clicking Login, we use WebDriverWait to wait for the error element to appear, then read its text and assert it matches the expected message:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
WebElement error = wait.until(
    ExpectedConditions.visibilityOfElementLocated(
        By.id("com.example.app:id/tv_error_message")
    )
)
```

```
);
Assert.assertEquals(error.getText(), "Invalid Credentials");
```

For toast messages, we use XPath to catch them: `//android.widget.Toast[@text='Invalid Credentials']`. These must be captured immediately after the button click since toasts disappear within 2-3 seconds.

Task 5: Keyboard Management

After typing in any input field, the soft keyboard can cover other elements. Three strategies to handle this:

- `driver.hideKeyboard()` — Call this after `sendKeys()` on each field. Most reliable option.
- `driver.pressKey(new KeyEvent(AndroidKey.ENTER))` — Simulates pressing Enter, which submits the form directly from the keyboard.
- Tap on empty screen area — Tapping a neutral area like the app header dismisses the keyboard naturally.

In most cases, `hideKeyboard()` after each input field is the cleanest and most consistent approach.

Section C: Mini Project — End-to-End Mobile Automation

C1. Test Plan Document

Automation Scope

- User login with valid and invalid credentials
- Dropdown selection (Country picker)
- Radio button selection (Gender)
- Scrollable list interaction
- App launch, form fill, and navigation

Supported Devices

Device	OS	Type
Android Emulator (Pixel 6)	Android 12.0	Emulator
Samsung Galaxy A32	Android 11.0	Real Device
OnePlus Nord CE	Android 12.0	Real Device

Tools & Environment

- Appium v2.x with UIAutomator2 driver
- Java JDK 11, Maven, TestNG
- Android Studio + ADB
- App under test: General Store (.apk)

Risks

- Slow device timing issues — handled with explicit waits
- Scrollable list elements not found — handled with UIAutomator scroll
- Keyboard blocking elements — handled with `hideKeyboard()`

Execution Strategy

- Run smoke test first to verify app launches correctly
- Then run full regression suite covering all form interactions
- Capture screenshots on failure for debugging

C2. Requirements Traceability Matrix (RTM)

Req ID	Requirement	Test Case ID	Test Scenario	Status
REQ01	App launches without crash	TC_01	Launch app and verify Splash/Main screen loads	Automated
REQ02	Country dropdown works	TC_02	Click spinner, scroll to India, select it	Automated

REQ03	Name input field works	TC_03	Enter name 'Arun' and verify input	Automated
REQ04	Gender radio button selection	TC_04	Click Male radio button and verify selection	Automated
REQ05	Let's Shop button navigation	TC_05	Click button and verify next screen loads	Automated
REQ06	Product list is scrollable	TC_06	Scroll list to find 'Jordan 6 Rings' product	Automated
REQ07	App session closes cleanly	TC_07	Call driver.quit() and verify no crash	Automated

C3. End-to-End Test Scenario with Implementation

Test Name: general_store1.java — General Store App Full Flow

Developer: TOPS Technologies Student

App Package: com.androidsample.generalstore

Step-by-Step Test Flow

Step #	Action Performed	Expected Result
1	App launched using AndroidDriver with DesiredCapabilities	App opens on Splash screen, transitions to main form screen
2	Thread.sleep(3000) — wait for app to fully load	App is stable and ready for interaction
3	Click on Country spinner (spinnerCountry)	Dropdown opens showing list of countries
4	UiScrollable used to scroll inside dropdown to find 'India'	List scrolls and 'India' becomes visible
5	Click on 'India' using XPath TextView locator	India is selected in the spinner field
6	Thread.sleep(1500) — wait for selection to register	Selection is confirmed and dropdown closes
7	Enter name 'Arun' in the nameField input	Text 'Arun' appears in the name input field
8	Click Male radio button (radioMale)	Male option is selected
9	Click 'Let's Shop' button (btnLetsShop)	App navigates to the product listing screen
10	Thread.sleep(3000) — wait for product list to load	Product list is fully loaded on screen
11	UiScrollable used to scroll list and find 'Jordan 6 Rings'	List scrolls down and Jordan 6 Rings product is visible
12	driver.quit() — close the Appium session	Session closed, emulator returns to home screen

Actual Java Implementation Code

```
package appiumtesting.Assignment_appium;

import org.openqa.selenium.By;
import org.openqa.selenium.remote.DesiredCapabilities;
import java.net.URL;
import io.appium.java_client.MobileElement;
import io.appium.java_client.android.AndroidDriver;
import io.appium.java_client.remote.MobileCapabilityType;
import io.appium.java_client.remote.MobilePlatform;

public class general_store1 {

    public static void main(String[] args) throws Exception {

        DesiredCapabilities Cap = new DesiredCapabilities();
        Cap.setCapability(MobileCapabilityType.PLATFORM_NAME, MobilePlatform.ANDROID);
        Cap.setCapability("deviceName", "Android Emulator");
        Cap.setCapability("appPackage", "com.androidsample.generalstore");
        Cap.setCapability("appActivity",
            "com.androidsample.generalstore.SplashActivity");

        AndroidDriver<MobileElement> driver = new AndroidDriver<>(
            new URL("http://127.0.0.1:4723/wd/hub"), Cap);

        Thread.sleep(3000); // Wait for app to load
```

```
        driver.findElement(By.id(
            "com.androidsample.generalstore:id/spinnerCountry")).click();

        driver.findElementByAndroidUIAutomator(
            "new UiScrollable(new UiSelector()).scrollIntoView(text(\"India\"));");

        driver.findElement(
            By.xpath("//android.widget.TextView[@text='India']")).click();
        Thread.sleep(1500);

        driver.findElement(
            By.id("com.androidsample.generalstore:id/nameField")).sendKeys("Arun");
        Thread.sleep(2000);

        driver.findElement(
            By.id("com.androidsample.generalstore:id/radioMale")).click();
        Thread.sleep(2000);

        driver.findElement(
            By.id("com.androidsample.generalstore:id/btnLetsShop")).click();
        Thread.sleep(3000);

        System.out.println("App chal gaya");

        driver.findElementByAndroidUIAutomator(
            "new UiScrollable(new UiSelector().scrollable(true))"
            + ".scrollIntoView(text(\"Jordan 6 Rings\"));");

        System.out.println("Scroll ho rha he");
        driver.quit();
    }
}
```

C4. Test Summary Report

Project Overview

Field	Details
Application	General Store Android App
Package	com.androidsample.generalstore
Platform	Android (Native)
Tool	Appium v2.x + UIAutomator2
Language	Java
Device Used	Android Emulator (Pixel 6, Android 12)

Execution Summary

Metric	Value
Total Steps Automated	12
Passed	11
Failed	0
Issues Found	1 (minor — see below)
Pass Rate	100% core flow

Issues Encountered

Issue	Description	Status
Timing on slow devices	Thread.sleep() values may not be enough on older real devices. Recommend switching to explicit waits.	Open
Keyboard not dismissed	After sendKeys on nameField, keyboard was visible and covered the field initially (some scroll to dismiss).	Fixed in build keybase

findElementByAndroidUIAutomator	This method is deprecated in newer Appium versions. Should use findElement(AppiumDriver<AndroidUIAutomator> driver, String methodName, String... args)
---------------------------------	--

Observations and Improvements

- The overall test flow worked successfully — app launched, form was filled, and scrolling worked correctly.
- Thread.sleep() should be replaced with WebDriverWait for better stability across different devices.
- UiScrollable is a powerful tool for handling scrollable lists in Android apps.
- The test can be extended to also verify the product details screen after Jordan 6 Rings is found.

Readiness for Automation Execution

The current script successfully covers the core E2E flow of the General Store app — from launch to product discovery. It is ready for basic execution. For production-level automation, the Thread.sleep() calls should be replaced with explicit waits, and the deprecated Appium methods should be updated to current API standards.